StudyAI

Copyright Registration Supporting Document, Copyright Act, 1957 (India)

Author: Neelkamal Gupta · Date: April 2026

GitHub: github.com/neelkamal1969/quiz-project · Live: quiz-project-five-inky.vercel.app

Author: Neelkamal Gupta (Single author)

Nature of Work: Computer Programme - Literary Work under Section 2(o) read with Section 2(ffc) of the Copyright Act, 1957

Language / Framework: JavaScript (React.js / JSX, Node.js / Express)

Stack: Frontend: React + Tailwind CSS | Backend: Node.js, Express, NeonDB (PostgreSQL) | AI: Google Gemini API

Date of Creation: April 2026

Deployment: Live web application (Vercel + Render)

This document presents the original source code of StudyAI for the purpose of copyright registration. All code was written by the author. The selections below represent the most architecturally significant and expressive portions of the work.

OWNER MEMBERS DETAILS

NAME	ADDRESS WITH EMAIL AND MOBILE NO.
Neelkamal Gupta	**Address: Shimlapuri, LDH-141003(PB-IN)** **Official Email: neelkamal1969.be22@chitkara.edu.in** **Personal Email: neel76441969@gmail.com** **Mobile No.: +91 7527017902**

1. Backend - index.js (Node.js / Express)

Core server file. Handles all API routes, AI integration, authentication, rate limiting, and database access.

1.1 Middleware & Security Setup

```
const app = express();
app.use(helmet());
app.use(cors());
app.use(express.json());

const generalLimiter = rateLimit({
    windowMs: 15 * 60 * 1000,
    max: 100,
    message: { error: "Too many requests. Please try again later." }
});
app.use(generalLimiter);

const aiLimiter = rateLimit({
    windowMs: 60 * 1000,
    max: 10,
    message: { error: "Too many AI requests. Please wait 60 seconds." }
});
```

1.2 Safe Gemini JSON Parser

```
const parseGeminiJSON = (text) => {
    try {
        let clean = text.replace(/```json|```/g, '').trim();
        const match = clean.match(/\[[\s\S]*\]/);
        if (match) clean = match[0];
        const parsed = JSON.parse(clean);
        return Array.isArray(parsed) ? parsed : [parsed];
    } catch (err) {
        throw new Error("AI did not return valid JSON");
    }
};
```

1.3 Personalised AI Question Generation - POST /page

```
app.post('/page', aiLimiter, [
    body('userId').optional().isInt(),
    body('pageText').trim().notEmpty()
], async (req, res) => {
    const { pageText, userId } = req.body;
    const users = await sql`
        SELECT age, gender, degree, difficulty, question_level
        FROM users WHERE id = ${userId}`;
    const profile = users[0] || {};

    const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
    const prompt = `Act as a Personal Academic AI Tutor.
        USER CONTEXT:
        - Age: ${profile.age || 'Not specified'}
        - Degree: ${profile.degree || 'General Education'}
        - Knowledge Level: ${profile.difficulty || 'Intermediate'}
        - Question Style: ${profile.question_level || 'Conceptual'}
```

```
        TASK: Analyse the source material and generate all
        possible high-quality Q&A pairs as a JSON array.`;

    const result = await model.generateContent(prompt);
    const data = parseGeminiJSON(result.response.text());
    res.json(data);
});
```

1.4 Auth - Signup & Login

```
app.post('/signup', [
    body('email').isEmail().normalizeEmail(),
    body('password').isLength({ min: 6 })
], async (req, res) => {
    const { email, password } = req.body;
    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);
    const newUser = await sql`
        INSERT INTO users (email, password)
        VALUES (${email}, ${hashedPassword})
        RETURNING id, email`;
    res.status(201).json({ message: "User created", user: newUser[0] });
});

app.post('/login', [...], async (req, res) => {
    const { email, password } = req.body;
    const user = (await sql`SELECT * FROM users WHERE email = ${email}`)[0];
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(401).json({ error: "Invalid credentials." });
    const token = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '24h' });
    res.json({ token, user: { id: user.id, email: user.email } });
});
```

1.5 Vault - Save, Fetch, Delete Cards

```
app.post('/save-card', [...], async (req, res) => {
    const { userId, question, answer } = req.body;
    const saved = await sql`
        INSERT INTO saved_cards (user_id, question, answer)
        VALUES (${userId}, ${question}, ${answer})
        RETURNING id`;
    res.status(200).json({ message: "Card saved!", id: saved[0].id });
});

app.get('/vault/:userId', ..., async (req, res) => {
    const savedCards = await sql`
        SELECT * FROM saved_cards
        WHERE user_id = ${req.params.userId}
        ORDER BY created_at DESC`;
    res.json(savedCards);
});

app.delete('/vault/:cardId', ..., async (req, res) => {
    await sql`DELETE FROM saved_cards WHERE id = ${req.params.cardId}`;
    res.json({ message: "Card removed" });
});
```

1.6 Admin Logs - IST Timezone Query

```
app.get('/admin/logs', async (req, res) => {
    const logs = await sql`
        SELECT id, email, topic,
            search_time AT TIME ZONE 'UTC'
                        AT TIME ZONE 'Asia/Kolkata' as search_time
        FROM user_searches
        ORDER BY search_time DESC
        LIMIT 100`;
    res.json(logs);
});
```

2. Local Algorithms - PhotoToQuestions.jsx

Original implementations of Bloom's Taxonomy classifier, Flesch-Kincaid readability scorer, keyword extractor, and custom SVG data visualisations - all written from scratch without external libraries.

2.1 Bloom's Taxonomy Classifier

```
const BLOOMS = [
  { level: 'Remember',   color: '#6366f1', keywords:
['define','list','recall','name','identify','state'] },
  { level: 'Understand', color: '#0ea5e9', keywords:
['explain','summarize','classify','compare','interpret'] },
  { level: 'Apply',      color: '#10b981', keywords:
['use','solve','demonstrate','calculate','execute'] },
  { level: 'Analyze',    color: '#f59e0b', keywords:
['analyze','differentiate','examine','contrast','test'] },
  { level: 'Evaluate',   color: '#ef4444', keywords:
['evaluate','judge','justify','critique','assess'] },
  { level: 'Create',     color: '#a855f7', keywords:
['create','design','compose','construct','plan','generate'] },
];

function classifyQuestion(q) {
  const lower = q.toLowerCase();
  for (const b of BLOOMS)
    if (b.keywords.some(k => lower.includes(k))) return b;
  return BLOOMS[1];
}
```

2.2 Flesch-Kincaid Readability Scorer

```
function fleschKincaid(text) {
  const sentences = Math.max(1,
    text.split(/[.!?]+/).filter(s => s.trim().length > 3).length);
  const words = text.trim().split(/\s+/).filter(Boolean);
  const wc = Math.max(1, words.length);
  const syllables = words.reduce((a, w) => a + cntSyl(w), 0);
  const score = Math.max(0, Math.min(100,
    206.835 - 1.015 * (wc / sentences) - 84.6 * (syllables / wc)));
  const s = Math.round(score);
  const [label, color] =
    s >= 70 ? ['Easy', '#10b981'] :
    s >= 50 ? ['Moderate', '#f59e0b'] :
    s >= 30 ? ['Difficult', '#ef4444'] :
              ['Very Hard', '#a855f7'];
  return { score: s, label, color, wordCount: wc, sentenceCount: sentences };
}

function cntSyl(word) {
  word = word.toLowerCase().replace(/[^a-z]/g, '');
  if (word.length <= 3) return 1;
  word = word.replace(/(?:[^laeiouy]es|ed|[^laeiouy]e)$/, '');
  const m = word.match(/[aeiouy]{1,2}/g);
  return m ? m.length : 1;
}
```

2.3 Keyword Extractor (TF-based, no library)

```
function extractKeywords(text, n = 20) {
```

```
  const stop = new Set(['the','a','an','and','or','but','in','on',
    'at','to','for','of','with','by','is','was','are','were','be',
    'have','has','do','did','will','would','that','this','it','not']);
  const words = text.toLowerCase()
    .replace(/[^a-z\s]/g, ' ').split(/\s+/)
    .filter(w => w.length > 3 && !stop.has(w));
  const freq = {};
  words.forEach(w => { freq[w] = (freq[w] || 0) + 1; });
  return Object.entries(freq)
    .sort((a, b) => b[1] - a[1]).slice(0, n)
    .map(([word, count]) => ({ word, count }));
}
```

2.4 Custom SVG - Bloom's Taxonomy Sunburst Chart

```
function BloomSunburst({ data }) {
  const [hov, setHov] = useState(null);
  const total = data.reduce((s, d) => s + d.count, 0) || 1;
  const cx = 100, cy = 100, r1 = 34, r2 = 82;
  let angle = -Math.PI / 2;
  const slices = data.map(d => {
    const sweep = (d.count / total) * 2 * Math.PI;
    const a1 = angle, a2 = angle + sweep; angle = a2;
    const lg = sweep > Math.PI ? 1 : 0;
    const mk = (r, a) => [cx + r*Math.cos(a), cy + r*Math.sin(a)];
    const [x1,y1]=mk(r2,a1),[x2,y2]=mk(r2,a2),[x3,y3]=mk(r1,a2),[x4,y4]=mk(r1,a1);
    const mid = a1 + sweep/2;
    const [lx,ly] = mk((r1+r2)/2, mid);
    return { ...d,
      path: `M${x1},${y1} A${r2},${r2} 0 ${lg} 1 ${x2},${y2}
              L${x3},${y3} A${r1},${r1} 0 ${lg} 0 ${x4},${y4} Z`,
      lx, ly, sweep };
  });
  return (
    <svg width="200" height="220" viewBox="0 0 200 220">
      {slices.map((s,i) => (
        <g key={i} onMouseEnter={()=>setHov(i)} onMouseLeave={()=>setHov(null)}>
          <path d={s.path} fill={s.color}
            opacity={hov===null?0.82:hov===i?1:0.2}
            stroke="#fff" strokeWidth="2" />
        </g>
      ))}
      <circle cx={cx} cy={cy} r={r1-1} fill="white"/>
      <text x={cx} y={cy+10} textAnchor="middle" fontSize="17" fontWeight="900">
        {total}
      </text>
    </svg>
  );
}
```

2.5 Custom SVG - Readability Gauge

```
function ReadabilityGauge({ score, label, color }) {
  const cx=80, cy=72, r=54;
  const rad = ((score/100*180-90)*Math.PI/180);
  const fx = cx + r*Math.cos(rad), fy = cy + r*Math.sin(rad);
  const lg = score/100*180 > 180 ? 1 : 0;
  return (
    <svg width="160" height="96" viewBox="0 0 160 96">
```

```
      <path d={`M${cx-r},${cy} A${r},${r} 0 0 1 ${cx+r},${cy}`}
        fill="none" stroke="rgba(0,0,0,.07)" strokeWidth="11"
strokeLinecap="round"/>
      <path d={`M${cx-r},${cy} A${r},${r} 0 ${lg} 1 ${fx},${fy}`}
        fill="none" stroke={color} strokeWidth="11" strokeLinecap="round"/>
      <line x1={cx} y1={cy} x2={fx} y2={fy} stroke={color} strokeWidth="2.5"/>
      <circle cx={cx} cy={cy} r="4.5" fill={color}/>
      <text x={cx} y={cy+15} textAnchor="middle" fontSize="20" fontWeight="900">
        {score}
      </text>
    </svg>
  );
}
```

3. Interactive Flashcard Engine - ResultSection.jsx

3D flip flashcards with keyboard navigation, speech synthesis, confidence rating, search modal, and guest-aware save logic.

3.1 Auth Helper & Guest-Aware Save Button

```
const getLoggedInUser = () => {
  try {
    const token = localStorage.getItem('token');
    const user  = JSON.parse(localStorage.getItem('user') || 'null');
    if (token && user && user.id) return user;
    return null;
  } catch { return null; }
};

function SaveButton({ isSaved, isGuest, onClick }) {
  return (
    <button onClick={onClick}
      title={isGuest ? 'Create a free account to save cards'
            : isSaved ? 'Remove from vault' : 'Save to vault'}
      style={{
        border: isSaved ? '1.5px solid #10b981'
                : isGuest ? '1.5px solid rgba(99,102,241,0.3)'
                : '1.5px solid #e0e7ff',
        background: isSaved ? 'rgba(16,185,129,0.1)'
                    : isGuest ? 'rgba(99,102,241,0.06)' : 'white',
        color: isSaved ? '#10b981' : '#6366f1',
      }}>
      {isSaved ? 'Saved' : 'Save'}
    </button>
  );
}
```

3.2 3D Flip Card CSS Animation

```
.rs-scene { perspective: 1400px; }

.rs-card-inner {
  position: relative; width: 100%; height: 100%;
  transform-style: preserve-3d;
  transition: transform 0.55s cubic-bezier(0.4, 0.2, 0.2, 1);
  will-change: transform;
}
.rs-card-inner.rs-flipped { transform: rotateY(180deg); }

.rs-face {
  position: absolute; inset: 0;
  backface-visibility: hidden;
  -webkit-backface-visibility: hidden;
  border-radius: 20px; overflow: hidden;
}
.rs-face-back { transform: rotateY(180deg); }

@keyframes rs-slide-r {
  from { opacity: 0; transform: translateX(28px) scale(0.97); }
  to   { opacity: 1; transform: translateX(0)    scale(1);    }
}
```

```
@keyframes rs-slide-l {
  from { opacity: 0; transform: translateX(-28px) scale(0.97); }
  to   { opacity: 1; transform: translateX(0)     scale(1);    }
}
```

3.3 Keyboard Shortcut Handler

```
useEffect(() => {
  const onKey = (e) => {
    if (searchOpen || showOverview) return;
    switch (e.key) {
      case 'ArrowRight': e.preventDefault(); navigate('next'); break;
      case 'ArrowLeft':  e.preventDefault(); navigate('prev'); break;
      case ' ':          e.preventDefault(); setIsFlipped(f => !f); break;
      case 's': case 'S':
        if (isGuest) { setShowGuestBanner(true); }
        else if (isSaved) { handleUnsave(e, currentIndex); }
        else { handleSave(e, items[currentIndex], currentIndex); }
        break;
      case 'r': case 'R':
        speak(isFlipped ? items[currentIndex].answer
                        : items[currentIndex].question);
        break;
      case 'Escape':
        setShowShortcuts(false); setShowOverview(false);
        setSearchOpen(false);    setShowGuestBanner(false);
        break;
    }
  };
  window.addEventListener('keydown', onKey);
  return () => window.removeEventListener('keydown', onKey);
}, [currentIndex, isFlipped, isSaved, isGuest, items, searchOpen]);
```

3.4 Speech Synthesis Integration

```
const speak = (text) => {
  if (!('speechSynthesis' in window)) return;
  const synth = window.speechSynthesis;
  if (isSpeaking) { synth.cancel(); setIsSpeaking(false); return; }
  synth.cancel();
  const utt = new SpeechSynthesisUtterance(text);
  utt.rate = 0.94; utt.pitch = 1.06; utt.volume = 1;
  utt.onend   = () => setIsSpeaking(false);
  utt.onerror = () => setIsSpeaking(false);
  synth.speak(utt);
  setIsSpeaking(true);
};
```

4. Repositionable Navigation - Header.jsx

A floating navigation bar that can be docked to the top, left, or right edge. Includes a 3D tilt effect, animated hamburger menu, and a position picker widget.

4.1 Dock Position Config & 3D Tilt Effect

```
const NAV_ITEMS = [
  { to: '/myquestions', icon: '≡', fullLabel: 'Question Bank' },
  { to: '/valueInput',  icon: '◈', fullLabel: 'Topic Search'  },
  { to: '/imageInput',  icon: '◎', fullLabel: 'Image OCR'     },
  { to: '/profile-setup',icon: '⊙', fullLabel: 'Profile'      },
];
const DOCK_POSITIONS = ['top', 'left', 'right'];

// 3D tilt - only active when docked to top
const handleMouseMove = (e) => {
  if (dock !== 'top') return;
  const rect = navRef.current?.getBoundingClientRect();
  const cx = rect.left + rect.width / 2;
  const cy = rect.top  + rect.height / 2;
  const dx = (e.clientX - cx) / (rect.width  / 2);
  const dy = (e.clientY - cy) / (rect.height / 2);
  setTilt({ x: dy * -4, y: dx * 4 });
};

const tiltStyle = dock === 'top'
  ? { transform: `perspective(600px) rotateX(${tilt.x}deg) rotateY(${tilt.y}deg)` }
  : {};
```

4.2 Prismatic Glass Edge CSS

```
.nav-pill::before {
  content: '';
  position: absolute; inset: 0;
  border-radius: inherit;
  background: linear-gradient(
    135deg,
    rgba(255,255,255,0.55) 0%,
    rgba(99,102,241,0.08) 40%,
    rgba(14,165,233,0.06) 70%,
    rgba(255,255,255,0.3) 100%
  );
  pointer-events: none; z-index: 0;
}
```

5. Profile Setup - ProfileSetup.jsx

Multi-step onboarding form with animated constellation canvas background, degree auto-suggest, difficulty picker, and live completion percentage.

5.1 Constellation Canvas Background

```
const ConstellationBg = () => {
  const canvasRef = useRef(null);
  useEffect(() => {
    const canvas = canvasRef.current;
    const ctx = canvas.getContext('2d');
    let W = canvas.width  = window.innerWidth;
    let H = canvas.height = window.innerHeight;
    const COUNT = Math.min(50,
      Math.floor((window.innerWidth * window.innerHeight) / 18000));
    const nodes = Array.from({ length: COUNT }, () => ({
      x: Math.random() * W, y: Math.random() * H,
      vx: (Math.random()-0.5)*0.28, vy: (Math.random()-0.5)*0.28,
      r: 1.2 + Math.random()*1.8,
    }));
    const draw = () => {
      ctx.clearRect(0, 0, W, H);
      nodes.forEach(n => { n.x+=n.vx; n.y+=n.vy;
        if(n.x<0||n.x>W) n.vx*=-1;
        if(n.y<0||n.y>H) n.vy*=-1; });
      for (let i=0; i<nodes.length; i++)
        for (let j=i+1; j<nodes.length; j++) {
          const dx=nodes[i].x-nodes[j].x, dy=nodes[i].y-nodes[j].y;
          const dist = Math.sqrt(dx*dx+dy*dy);
          if (dist < 140) {
            ctx.beginPath();
            ctx.strokeStyle = `rgba(79,70,229,${0.11*(1-dist/140)})`;
            ctx.lineWidth = 0.8;
            ctx.moveTo(nodes[i].x, nodes[i].y);
            ctx.lineTo(nodes[j].x, nodes[j].y);
            ctx.stroke();
          }
        }
      requestAnimationFrame(draw);
    };
    draw();
  }, []);
  return <canvas ref={canvasRef} style={{ position:'fixed', inset:0, zIndex:0 }} />;
};
```

5.2 Profile Completion Calculator

```
const safe = (v) => (v == null ? '' : String(v));

const completionPct = (profile) => {
  const fields = ['age','gender','degree','difficulty','question_level','goal'];
  const filled = fields.filter(f => safe(profile[f]) !== '').length;
  return Math.round((filled / fields.length) * 100);
};
```

6. Admin Analytics Dashboard - AdminLogs.jsx

Real-time search activity tracking with KPI cards, SVG hourly bar chart, topic word cloud, and CSV / print-PDF export - all data computed client-side.

6.1 Analytics Computation

```
const computeAnalytics = (logs) => {
  const topicCount = {}, userCount = {};
  const hourBucket = Array(24).fill(0);
  const today = new Date().toDateString();
  let todayCount = 0;
  logs.forEach((l) => {
    const t = (l.topic || '').trim().toLowerCase();
    const e = (l.email || '').trim().toLowerCase();
    topicCount[t] = (topicCount[t] || 0) + 1;
    userCount[e]  = (userCount[e]  || 0) + 1;
    const h = new Date(l.search_time).getHours();
    hourBucket[h]++;
    if (new Date(l.search_time).toDateString() === today) todayCount++;
  });
  const topTopics = Object.entries(topicCount)
    .sort((a,b)=>b[1]-a[1]).slice(0,20);
  return { totalSearches: logs.length, uniqueUsers: Object.keys(userCount).length,
           uniqueTopics: Object.keys(topicCount).length, todaySearches: todayCount,
           topTopics, hourBucket };
};
```

6.2 SVG Hourly Activity Bar Chart

```
const HourlyChart = ({ hourBucket }) => {
  const max = Math.max(...hourBucket, 1);
  const W = 600, H = 80, barW = W / 24;
  const now = new Date().getHours();
  return (
    <svg viewBox={`0 0 ${W} ${H+24}`} style={{ width:'100%' }}>
      {hourBucket.map((v, h) => {
        const bh  = Math.max(4, (v/max)*H);
        const x   = h*barW + barW*0.15;
        const color = (h===now) ? '#4f46e5' : v>0 ? '#818cf8' : '#e2e8f0';
        return (
          <g key={h}>
            <rect x={x} y={H-bh} width={barW*0.7} height={bh}
              rx={3} fill={color} opacity={h===now ? 1 : 0.75}>
              <title>{`${h}:00 - ${v} search${v!==1?'es':''}`}</title>
            </rect>
            {(h%6===0) && (
              <text x={x+barW*0.35} y={H+16}
                textAnchor="middle" fontSize={9} fill="#94a3b8">
                {`${h}h`}
              </text>
            )}
          </g>
        );
      })}
    </svg>
  );
};
```

6.3 CSV Export Utility

```
const exportCSV = (logs) => {
  const header = ['Email', 'Topic', 'Time (IST)'];
  const rows = logs.map((l) => [
    `"${l.email}"`, `"${l.topic}"`, `"${fmtTime(l.search_time)}"`,
  ]);
  const csv = [header, ...rows].map((r) => r.join(',')).join('\n');
  const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' });
  const url  = URL.createObjectURL(blob);
  const a = Object.assign(document.createElement('a'), {
    href: url, download: `studyai_logs_${Date.now()}.csv`,
  });
  document.body.appendChild(a); a.click();
  document.body.removeChild(a); URL.revokeObjectURL(url);
};
```

7. Additional Components

7.1 Login - Inline Email Validation (LoginPage.jsx)

```
const validateEmail = (value) => {
  if (!value) { setEmailError('Email is required'); return false; }
  // RFC 5322-inspired pattern: checks local@domain.tld structure
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
  if (!emailRegex.test(value)) {
    setEmailError('Please enter a valid email address'); return false;
  }
  setEmailError(''); return true;
};
```

7.2 Vault - Smart Tag Cloud & Practice Mode (Vault.jsx)

```
const getTagCloud = () => {
  const text = cards.map(c => c.question.toLowerCase()).join(' ');
  const words = text.split(/\s+/).filter(w =>
    w.length > 3 && !['the','and','for','you','this','with','from'].includes(w));
  const freq = {};
  words.forEach(w => { freq[w] = (freq[w]||0)+1; });
  return Object.entries(freq)
    .sort((a,b)=>b[1]-a[1]).slice(0,12)
    .map(([word,count]) => ({ word, size: Math.max(14, 14+count*6) }));
};

// Interactive 3D flip card in Practice Mode
<div onClick={toggleFlip}
  style={{ perspective: '1200px', height: '460px', cursor: 'pointer' }}>
  <div style={{
    transformStyle: 'preserve-3d',
    transform: isFlipped ? 'rotateY(180deg)' : 'rotateY(0deg)',
    transition: 'transform 0.7s',
    position: 'relative', width: '100%', height: '100%',
  }}>
    {/* Front - Question */}
    <div style={{ backfaceVisibility: 'hidden', position: 'absolute', inset: 0 }}>
      <p>{practiceCards[practiceIndex].question}</p>
    </div>
    {/* Back - Answer */}
    <div style={{ backfaceVisibility: 'hidden', transform: 'rotateY(180deg)',
      position: 'absolute', inset: 0 }}>
      <p>{practiceCards[practiceIndex].answer}</p>
    </div>
  </div>
</div>
```

7.3 Homepage - Vault Export (CSV + PDF) Functions (HomePage.jsx)

```
const exportToCSV = () => {
  const headers = ['Question', 'Answer', 'Saved Date'];
  const rows = fullVaultCards.map(card => [
    `"${card.question.replace(/"/g, '""')}"`,
    `"${card.answer.replace(/"/g, '""')}"`,
    new Date(card.created_at).toLocaleDateString('en-IN')
  ]);
  const csvContent = [headers,...rows].map(r=>r.join(',')).join('\n');
```

```
  const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });
  const link = document.createElement('a');
  link.setAttribute('href', URL.createObjectURL(blob));
  link.setAttribute('download',
    `StudyAI-Vault-${new Date().toISOString().slice(0,10)}.csv`);
  document.body.appendChild(link); link.click(); document.body.removeChild(link);
};
```

Declaration of Authorship

I, Neelkamal Gupta, declare that:

1. I am the sole author of all source code presented in this document.
2. The code was written by hand over multiple development sessions. All architectural decisions, component structures, algorithm designs, and visual styling choices were made by me personally.
3. The work is original and has not been copied from any other source. Where standard programming patterns are used (e.g. rate limiting, bcrypt hashing), the specific selection, arrangement, integration, and application within this project constitute an original creative expression.
4. The work includes several algorithms implemented from scratch: Flesch-Kincaid readability scorer, Bloom's Taxonomy classifier, keyword frequency extractor, custom SVG data visualisations (sunburst chart, radial gauge, hourly bar chart), and animated canvas constellation background - none of which rely on external libraries.
5. This document is submitted in support of a copyright registration application under the Copyright Act, 1957 (India).

Name: Neelkamal Gupta

Date: April 2026